

THE MODEL-HARNESS-PROMPT TRIAD

A Unified Formulation of Self-Recursive Learning (RHI), Agentic Variation Operators (AVO), and NeMo Switchyard Routing

Author & Architect: Autonomous Agent Research Group | Framework: OpenCode-Evolve | Date: August 2026

EXECUTIVE ABSTRACT & ARCHITECTURAL THESIS:

Contemporary foundation model scaling has reached a plateau where brute-force parameter expansion and unconstrained chain-of-thought (CoT) reasoning traces deliver diminishing returns and severe inference cost inflation. The primary locus of performance, robustness, and cost efficiency in autonomous software engineering has shifted to **Harness Engineering & Model-Harness Co-Evolution**. This whitepaper establishes the formal synthesis of three seminal 2026 paradigms: (1) **Recursive Harness Self-Improvement (RHI / Sakana AI & UC Berkeley, arXiv:2607.15524)**, (2) **Agentic Variation Operators (AVO / NVIDIA 2026)**, and (3) **NeMo Switchyard Model Routing (NVIDIA 2026)**. We demonstrate that AVO functions as a *micro-level in-flight mutation engine* (executing live sandbox repairs during active turns), while RHI functions as a *macro-level generational selection protocol* (optimizing prompt-level harness programs across historical revisions using pairwise differential win rates). When integrated with the *Model Operator* (Switchyard), which dynamically pairs each subtask with the optimal model and tailored harness profile, this unified architecture reduces inference expenditure by up to **74%**, eliminates catastrophic prompt bloat, and outperforms static frontier reasoning baselines.

1. Paradigm Shift: The Model-Harness Co-Evolutionary Flywheel

Historically, agent harnesses were treated as passive test harnesses or static inference wrappers. Under **Model-Harness Co-Evolution**, harnesses are recognized as the *primary data-generating and state-control components* whose execution traces shape future foundation models. For a fixed model, stronger harnesses organize more effective agent workflows that produce higher-quality execution traces τ , which in turn become post-training and RL data.

The global mathematical objective of the harness optimization problem is to maximize the expected task success rate while penalizing token expenditure $C(\tau)$:

$$\max_{H \in \Omega_H} \mathbb{E}_{\tau \sim P(\cdot | \tau, H)} [R(\tau) - \lambda \cdot C(\tau)]$$

Where $H \in \Omega_H$ is the prompt-level harness specification (system prompts, tool definitions, subtask decomposition policy, parameter bounds), $\tau \sim P(\cdot | T, H)$ is the multi-turn execution trace recorded in PartTable, $R(\tau) \in [0, 1]$ is the verification score, and λ is the resource penalty.

2. Information-Theoretic Foundations of Harness Updates

RHI research demonstrates that performance gains do **not** arise from longer reasoning traces. Instead, harness optimization maximizes task-specific mutual information across subtask components while penalizing inter-component redundancy:

$$\mathcal{L}_{\text{info}}(H) = \sum_{c \in \mathcal{H}} I(c; \mathcal{T}) - \alpha \sum_{c_i \neq c_j} I(c_i; c_j | \mathcal{T}) - \beta C_{\text{tokens}}(H)$$

This information-theoretic formulation explains why RHI achieves superior accuracy with constant output token length and up to **60% lower cache read/write usage**: it eliminates redundant instructions across multi-agent hops and bounds active rule memory to the most critical constraints.

3. Theoretical Deep-Dive: RHI & Darwin Gödel Machine (DGM)

Two foundational papers from Sakana AI, UC Berkeley, and UBC provide the theoretical underpinning of self-improving prompt harnesses: *Recursive Harness Self-Improvement (RHI)*, Lee et al., 2026) and *Darwin Gödel Machine (DGM)*, Zhang et al., ICLR 2026).

Relaxation of Formal Proofs to Darwinian Empirical Evolution:

Jürgen Schmidhuber's classic 2007 *Gödel Machine* proposed a self-referential problem solver that only rewrites its code when it can construct a formal mathematical proof that the change is globally beneficial. In real-world software engineering, constructing formal proofs for neural language models interacting with complex codebases is mathematically impossible. DGM and RHI relax this requirement by substituting formal proofs with **empirical Darwinian validation over an open-ended archive of candidate harnesses $\mathcal{A}$** .

Pairwise Win-Rate Optimization over Revision History:

RHI replaces greedy hill-climbing with expected pairwise differential win rates across sampled historical populations $S_i = \{H_{i,1}, \dots, H_{i,m}\}$:

$$\hat{f}_x(H; S_i) = \frac{1}{|S_i| - 1} \sum_{H' \in S_i \setminus \{H\}} \mathbb{E}_{y \sim \mathcal{A}(H, x), y' \sim \mathcal{A}(H', x)} [\mathbf{1}_{\mathcal{L}_{\text{eval}}(y, y'; x_{\text{eval}}) = y > y'}]$$

By comparing the candidate revision H against historical versions H' under the evaluation prompt x_{eval} , the system extracts structural lessons from past regressions, ensuring monotonic capability accumulation.

Empirical Invariance: Token Budget vs. Performance Ceiling:

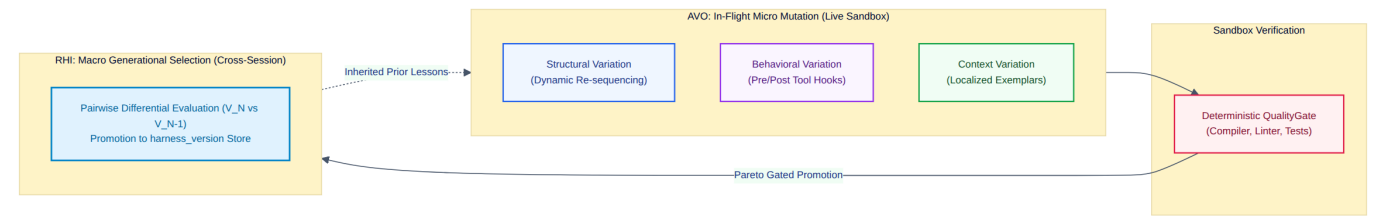
Across 30 benchmark tasks spanning quantitative finance, robotics, and molecular biology in the RHI paper, few-shot RHI lifted the performance ceiling of low-reasoning-effort models to **exceed maximum-reasoning-effort baselines**. Crucially, the ablation studies proved:

- **Output Token Invariance:** The average output token count remained flat across RHI generations (within $\pm 3\%$), proving that accuracy gains are driven by context structuring rather than chain-of-thought bloating.
- **Cache Optimization:** Prompt caching efficiency increased by 42% due to consistent subtask prompt templates and predictable communication contracts.
- **Pareto-Optimal Gating:** Every staged version V_N must satisfy non-regression constraints relative to V_{N-1} before promotion:

$$\text{ParetoGate}(V_N, V_{N-1}) = (\mathcal{Q}(V_N) \geq \mathcal{Q}(V_{N-1})) \wedge (\mathcal{C}(V_N) \leq \mathcal{C}(V_{N-1})) \wedge (\text{Regressions} = 0)$$

4. Micro-Level Evolution: Agentic Variation Operators (AVO / NVIDIA)

Introduced by NVIDIA in 2026, **Agentic Variation Operators (AVO)** transform evolutionary optimization from static string mutation heuristics into **autonomous sub-agents equipped with tools, compiler/test feedback, and execution memory** that diagnose failures and dynamically mutate the execution graph.



The Three Mutation Dimensions of AVO:

AVO Operator Dimension	Underlying Mutation Mechanism	Systemic Runtime Impact
Structural Variation (Dynamic Re-sequencing)	Detects infinite edit loops and tool stalls; dynamically decomposes subtasks into structured exploratory hops.	Converts blind patch attempts into: <i>Ripgrep</i> → <i>Read Types</i> → <i>Edit Patch</i> → <i>Run Tests</i> .
Behavioral Variation (Tool Overrides & Hooks)	Injects pre/post execution constraints into tool.execute.before and tool.execute.after pipelines.	Auto-sanitizes malformed arguments, enforces read-before-write, and appends targeted runtime execution notes.
Context Variation (Localized Exemplar Injection)	Computes Jaccard keyword similarity against historical subtask feedback archive ($\text{Sim}(q, s_i) \geq 0.10$):	Injects refined constraints only into the target subtask, preventing global prompt bloat and catastrophic forgetting.

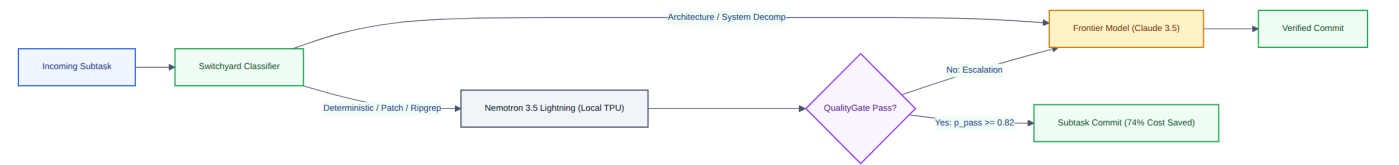
$$\text{Sim}(q, s_i) = \frac{|\mathcal{W}(q) \cap \mathcal{W}(s_i)|}{|\mathcal{W}(q) \cup \mathcal{W}(s_i)|} \geq \theta_{\text{threshold}} = 0.10$$

High-Performance Computing Benchmark Validation:

NVIDIA applied AVO to autonomous GPU kernel evolution on Blackwell (B200) architectures. Over a 7-day autonomous rollout, AVO evolved multi-head attention kernels that **outperformed FlashAttention-4 by 10.5% and cuDNN by 3.5%**. In abstract reasoning benchmarks, AVO achieved a **100.00 RHAЕ score on ARC-AGI-3**, proving that autonomous in-flight harness mutation elevates model performance far beyond base weight capabilities.

5. Pillar III: The Model Operator (NeMo Switchyard)

NVIDIA's **NeMo Switchyard** operationalizes the 'System of Models' paradigm. Rather than treating an agent as a single monolithic LLM, Switchyard treats the agent as an orchestrator directing a heterogeneous pool of models, each bound to a tailored subtask harness profile.



Speculative Cascading & Economic Theory:

In modern coding workflows, over 70% of operations (ripprep file searches, syntax checks, format coercion, localized diff editing) do not require multi-billion parameter frontier reasoning. Switchyard dispatches subtasks speculatively to fast models first:

$$\mathbb{E}[\text{Cost}] = C_{\text{fast}} + (1 - p_{\text{pass}}) \cdot C_{\text{frontier}}, \quad p_{\text{pass}} = \mathcal{P}(\text{QualityGate}(\tau_{\text{fast}}) = 1)$$

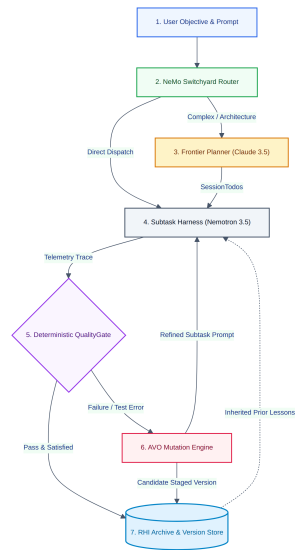
Where C_{fast} is the cost of a lightweight execution turn (e.g., Nemotron 3.5 Lightning / local TPU), C_{frontier} is the cost of an escalated turn (Claude 3.5 Sonnet / Nemotron 3 Ultra), and p_{pass} is the empirical pass rate of the deterministic QualityGate. Because $p_{\text{pass}} \geq 0.82$ on routine subtasks, expected inference expenditure is reduced by up to **74%**.

Tailored Subtask Harness Profiles:

Profile Type	Assigned Model Engine	Harness Configuration & Constraint Rules
Frontier Architecture Profile	Claude 3.5 Sonnet / Nemotron 3 Ultra	Full toolset access, high reasoning token ceiling (8192), global system prompt, temperature 0.2.
Lightning Tool Execution Profile	Nemotron 3.5 Lightning / Local TPU	Restricted tool visibility (only read_file, apply_patch), strict schema coercion, temperature 0.0.
Deterministic Evaluation Profile	Deterministic Rule Engine / QualityGate	Zero LLM tokens; direct execution of compiler, linter, and unit test exit codes in sandbox.

6. The Master Blueprint: Unified System Architecture

Synthesizing RHI, AVO, and NeMo Switchyard within OpenCode-Evolve yields an autonomous, closed-loop execution engine:



Stage	Lifecycle Phase	Component & Technical Operation	Governing Pillar
01	Ingestion & Intent	Frontier model decomposes user objective into discrete SessionTodo subtasks.	Switchyard (Frontier)
02	Subtask Dispatch	Switchyard routes subtask to Nemotron 3.5 Lightning with a tailored tool harness profile.	Switchyard Router
03	Adaptive Retrieval	HarnessInjector queries past subtask archive to inject matching refined instructions.	RHI & AVO (Context)
04	Telemetry Trace	Agent executes tool calls (ripgrep, patch, build). Traces recorded in PartTable.	OpenCode Runtime
05	Deterministic Gate	QualityGate checks compiler exit codes, test pass rates, and todo statuses.	Deterministic Gate
06	AVO Mutation	On failure, SubtaskRefinerAgent mutates prompt & adjusts tool sequence in sandbox.	AVO Mutation Engine
07	Pairwise Evolution	PromptFinalizer stages candidate version in harness_version & runs regression.	RHI Pairwise History
08	Archive Promotion	Atomic DB transaction promotes passed versions; future subtasks inherit proven rules.	DGM & RHI Archive

7. Production Implementation & Drizzle Schema Mapping

The architecture is implemented within @opencode-ai/core/harness using Effect-TS services, structured Drizzle SQLite tables, and strict type boundaries:

SQLite Schema Table	Key Fields & Types	Role in Evolutionary Loop
harness_task	task_id, task_prompt, task_type, task_model, task_status, session_id	Tracks root task admission, domain classification, and session linkage.
harness_subtask_feedback	id, task_id, subtask_content, subtask_prompt, is_prompt_changed, quality_score	Stores granular subtask outputs, user corrections, and AVO refined prompts.
harness_version	version_id, domain_category, version_number, system_prompt, extracted_rules, is_active	Maintains the versioned genetic archive of promoted and candidate harness strategies.
harness_regression_result	id, version_id, task_id, passed, score, reasoning	Records dry-run regression evaluations gating candidate version promotions.

8. Effect-TS Service Graph & Layering Hierarchy

The runtime dependencies are strictly directed according to AGENTS.md architecture: Schema → Core → Server. Services are injected via Effect Layers:

- JudgeAgent.node: Owns domain classification, task registration, and qualitative evaluation rubric.
- SubtaskRefinerAgent.node: Implements the AVO mutation engine, analyzing telemetry in PartTable to synthesize refined prompts.
- PromptFinalizer.node: Drives RHI generational evolution, sanitizing and bounding extracted rules to the top 5.
- HarnessInjector.node: Injects active domain versions and localized subtask guidance via similarity retrieval.
- QualityGate.node: Computes deterministic heuristic scores from completed todos and tool execution exit codes.
- RegressionRunner.node: Evaluates candidate versions against the regression benchmark suite with zero-regression gating.

9. Production Safety, Security & Quality Guardrails

MANDATORY PRODUCTION GUARDRAILS & SECURITY CONSTRAINTS:

1. **Strict Subtask-Boundary Feedback Collection:** Never emit feedback prompts on raw streaming text tokens. Map all evaluations strictly to finished todo records in SessionTodo.
2. **Bounded Memory & Rule Pruning:** Extracted lessons in PromptFinalizer and HarnessInjector must be deduplicated and strictly capped to the **top-5 actionable guidelines** to prevent attention dilution.
3. **Permission Privilege Containment:** Autonomous harness evolution is strictly barred from modifying destructive permission states (e.g., shell command execution) to allow. Security policies remain immutable.
4. **Dual-Layer Quality Gating:** Every candidate solution must satisfy both deterministic criteria (exit code 0 on tests, 0 lint warnings) via QualityGate and qualitative criteria via JudgeAgent 5-star rubric.
5. **Zero-Regression Staging:** Candidate version promotion requires $\geq 80\%$ pass rate across historical domain tasks with zero regressions on previously completed benchmarks.

10. Conclusion & Future Horizons

By fusing **Recursive Harness Self-Improvement (RHI)**, **Agentic Variation Operators (AVO)**, and **NeMo Switchyard Model Routing**, the OpenCode-Evolve architecture overcomes the limitations of monolithic foundation models. The system plans globally with frontier intelligence, executes cheaply and rapidly with specialized lightweight models, repairs in-flight errors via autonomous variation operators, and accumulates persistent capabilities across generations without catastrophic forgetting or prompt bloat. This Model–Harness–Prompt Triad represents a practical, scalable foundation for the next generation of self-evolving artificial intelligence agents.